

(a)

```
In [2]: import numpy as np
        from scipy.optimize import curve_fit
        import matplotlib.pyplot as plt

        t = np.array([0, 10, 20, 30, 40, 50])
        N = np.array([1000, 853, 727, 619, 527, 448])

        def exp_decay(t, N0, lambda_):
            return N0 * np.exp(-lambda_ * t)

        params, covariance = curve_fit(exp_decay, t, N, p0=[1000, 0.02])
        N0_fit, lambda_fit = params

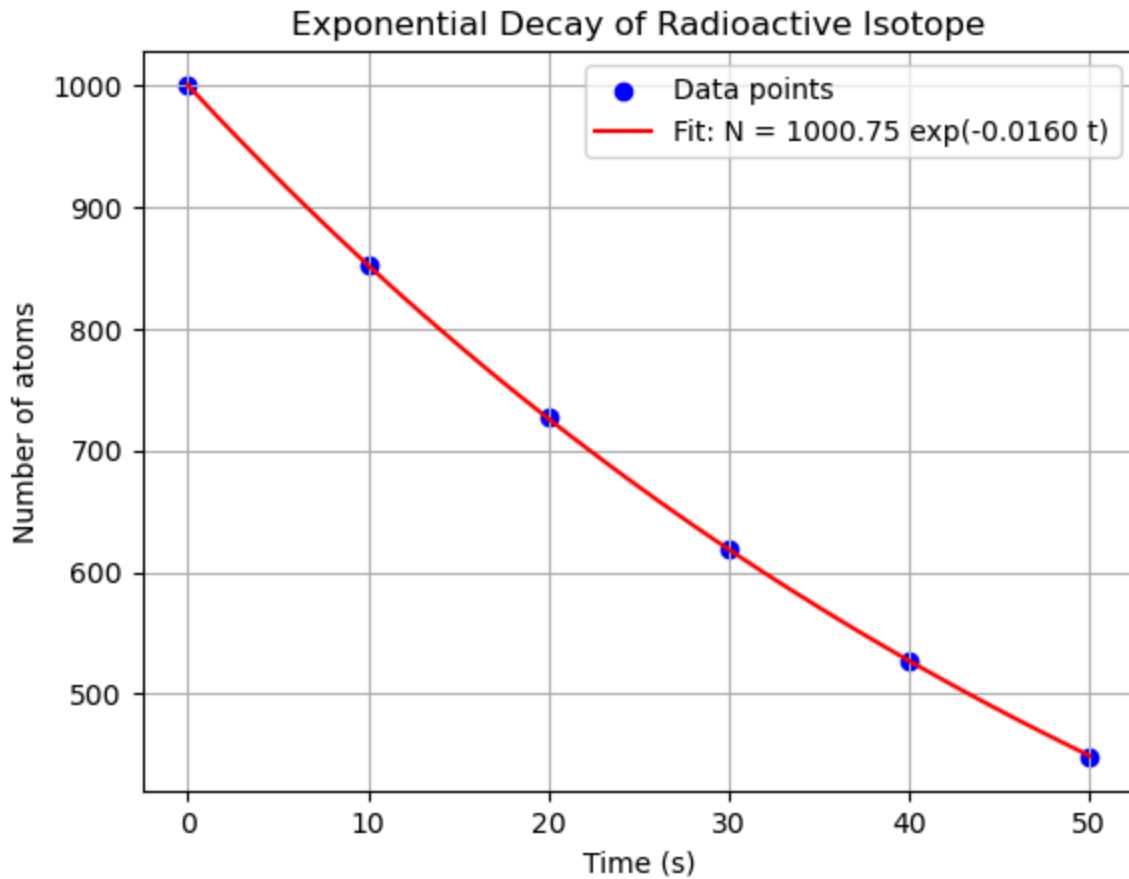
        print(f"Initial number of atoms (N0): {N0_fit:.2f}")
        print(f"Decay constant ( $\lambda$ ): {lambda_fit:.4f}")

        t_fit = np.linspace(0, 50, 100)
        N_fit = exp_decay(t_fit, N0_fit, lambda_fit)

        plt.scatter(t, N, color='blue', label='Data points')
        plt.plot(t_fit, N_fit, color='red', label=f'Fit: N = {N0_fit:.2f} exp(-{lambda_fit:.4f} * t)')
        plt.xlabel('Time (s)')
        plt.ylabel('Number of atoms')
        plt.title('Exponential Decay of Radioactive Isotope')
        plt.legend()
        plt.grid(True)
        plt.show()
```

Initial number of atoms (N0): 1000.75

Decay constant (λ): 0.0160



(b)

```
In [3]: import numpy as np
from scipy.optimize import curve_fit

t = np.array([0, 10, 20, 30, 40, 50])
N = np.array([1000, 853, 727, 619, 527, 448])

def exp_decay(t, N0, lambda_):
    return N0 * np.exp(-lambda_ * t)

params, covariance = curve_fit(exp_decay, t, N, p0=[1000, 0.02])
N0_fit, lambda_fit = params

N_pred = exp_decay(t, N0_fit, lambda_fit)

residuals = N - N_pred
ss_res = np.sum(residuals**2)
ss_tot = np.sum((N - np.mean(N))**2)
r_squared = 1 - (ss_res / ss_tot)

print(f"R-squared: {r_squared:.4f}")
```

R-squared: 1.0000